Solution Architecture

Wandr AI – AI Travel Itinerary Planner

1. Architecture Overview

The application follows a **three-tier architecture**: a React SPA frontend hosted on Vercel, a FastAPI Python backend on Render, and Supabase (PostgreSQL + Auth) as the data and identity layer. The Groq AI API is an external service consumed exclusively by the backend.

2. Component Diagram

Layer	Component	Technology	Hosted On
Presentation	React SPA (Wandr AI UI)	React 18 + Vite + Tailwind	Vercel
Presentation	Supabase Auth Client	Supabase JS ^2.45	Browser (Vercel-served)
Application	FastAPI REST API	Python 3.12 + FastAPI 0.115	Render
Application	JWT Middleware (security.py)	PyJWT + JWKS	Render (inline)
Application	AI Service (ai_service.py)	Groq SDK 0.11 + Tenacity	Render (inline)
Application	Budget Service	Pure Python	Render (inline)
Data	trips table	Supabase PostgreSQL	Supabase cloud
Data	users (auth.users)	Supabase Auth	Supabase cloud
External	Groq AI API	openai/gpt-oss-120b	Groq cloud

3. Security Architecture

Layer	Control	Implementation
Transport	HTTPS everywhere	Vercel + Render enforce TLS
Auth	JWT verification	PyJWT via Supabase JWKS (RS256) with HS256 fallback
CORS	Origin allowlist (regex)	allow_origin_regex in Starlette CORSMiddleware
Data	Row Level Security	Supabase RLS: auth.uid() = user_id on all policies

Secrets	Server-side only	GROQ_API_KEY, service_role_key, JWT_SECRET in Render env; never in VITE_ vars

4. Deployment Topology

Frontend URL	https://ai-travel-itinerary-planner-frontend-i97y5pqrz.vercel.app
Backend URL	https://ai-travel-itinerary-planner-q6j8.onrender.com
API Docs (Swagger)	https://ai-travel-itinerary-planner-q6j8.onrender.com/docs
Health Check	https://ai-travel-itinerary-planner-q6j8.onrender.com/api/health
Database	Supabase: https://ilhxiwvreknixzfnsxpv.supabase.co
AI Provider	Groq – console.groq.com

5. CORS Strategy

The backend uses Starlette's allow_origin_regex with an anchored pattern that matches: (a) localhost on any port for local development, (b) the stable production Vercel URL, and (c) all Vercel preview URLs whose subdomain starts with the project slug followed by a hash. This prevents any new preview deployment from triggering a backend change.